COURSE 1, AREAS 1, RUN 2 — the three files the router reads besides the grids

The course, the water it may not cross, and the settings for one run. `docs/web/route/course.js` is the reference: **what it refuses, these formats do not allow**, and where this document and that file differ, the file is right.

Why three files and not one

They have three different lifetimes, and a file's lifetime is the honest reason to split it.

	what it is	how often it changes
course	the race, as the sailing instructions give it	once, and then never
areas	the water: coastline, traffic schemes, exclusion zones	when the world or the source data changes
run	this experiment: which boat, which start, what to scale	every run

Put the run settings inside the course and one course file stops serving every boat and every start, which is the thing Larry's class-start decision requires. Put the areas in the course and a second race over the same water copies them.

COURSE 1

Plain text. Blank lines and lines beginning # are ignored anywhere. Values are separated by spaces or tabs. `\r\n` reads the same as `\n`. Positions are decimal degrees, latitude then longitude, north and east positive.

```
COURSE 1
name Example race 2025
source Invented for this document. Not a real course.
start line 50.76167 -1.29750 bearing 190 length 0.20 cross south
start class Class Zero 2025-05-24T10:10Z
start class Class One  2025-05-24T10:00Z
mark Outer Rock       50.18083 -4.25833 port
mark Return Buoy      50.70833 -1.47500 starboard
finish line 50.70833 -1.47500 bearing 000 length 0.10 cross north
end
```

The figures above are invented, and the file says so on its `source` line. An example named after a real race is taken for that race: PILOT checked these positions against the 2025 sailing instructions and found the finish 4.4 nm from where it belongs, which is exactly the fault `source` exists to prevent - both numbers are legal and no reader can tell. A document cannot carry a course that can be checked against its instructions, because the document is not the file; so this one carries a course that nobody

can mistake for data. **The real 2025 course is the course repository's own file**, where every figure sits beside the sailing-instruction text it came from.

The header and the body

Line	Form	Notes
`COURSE 1`	—	Must be the first line. The version is the whole line.
`name`	free text	One line. What a reader calls this course.
`source`	free text	**Required.** Where every figure below came from.
`start`	see below	One `line`, and one or more `class`.
`mark`	name lat lon side	In the order they are rounded. At least one.
`finish`	see below	Exactly one.
`end`	—	**Last line.** Nothing is read after it.

`end` exists for the same reason `rows` exists in GRID 1: a text file's realistic failure is a copy that stopped. A course that is truncated after its second mark is a course the router will happily sail.

The order of the marks is the order of the lines

There is no index number on a mark. A number beside a line whose position already carries the same information is a second copy of it, and the two can disagree; the file order is the order. A reader that wants to renumber can count.

The side is required, and may be `either`

`port`, `starboard` or `either`. **It is required to be present** even when it is `either`, because the difference between "this mark has no required side" and "nobody wrote the side down" is exactly the difference a router cannot see and a navigator can.

Why the router needs it at all. An isochrone router expands a frontier of reachable positions and takes the fastest. Given only the order, the fastest way past the Return Buoy may be the wrong side of it, and the result is a perfectly convincing track that would be scored as not having sailed the course. The side is not a check applied afterwards; it constrains the search.

What it does not carry: how close counts as rounded. That is a property of the run, not of the race — the sailing instructions do not give a radius — so it lives in the run file as `round`.

Lines are given the way the sailing instructions give them

Two forms, each named in the file, because the instructions use both:

```
start  line <lat> <lon> bearing <deg> length <nm> cross <side>
start  line <lat1> <lon1> to <lat2> <lon2> cross <side>
```

The second is two ends. The first is one end, a true bearing and a length in nautical miles, which is how an instruction states a line with only one end published — as the 2025 start is, from Williams Shipping.

Why both, when GRID 1 insists on one of everything. GRID 1 fixes units and sign because a file in the other convention is *silently* wrong and nothing says so. These two forms are not that: each is self-describing and each converts exactly to the other. What is at stake here is different — if the file must hold two endpoints, then whoever writes it performs a conversion, and the file no longer resembles the document it came from. Nobody can then check the course against the sailing instructions by eye, which is the one check that matters most and the only one a human can do. **The file should look like its source.** The reader does the trigonometry, once, where it can be tested.

`cross` is the side the course lies on: the direction a boat crosses the line going the right way. `north`, `south`, `east`, `west`, or a bearing in degrees. It is required on both lines — a line without a sense can be crossed to finish before you have started.

Every class start is listed; the run picks one

```
start class Class Zero 2025-05-24T10:10Z
start class Class One  2025-05-24T10:00Z
```

Larry's decision, 10 November: the router runs the class start as a scenario. So the course names every start in SI 8.1 and fixes none of them, and the run file chooses — by class name, or by a time of its own for a "what if". Ten minutes at Cowes can be hours at St Alban's Head, and that difference is the lesson.

The course file names no boat. The boat is a property of the run.

What does not go in a course file

Teaching waypoints do not. practical-navigator asked whether gates could be allowed — a point the router must pass at a chosen distance off St Alban's Head or Portland Bill. The answer is no, and the reason is worth stating plainly, because the need behind the question is real.

A course file's value is that it can be checked, line by line, against the sailing instructions. The moment it also holds things that are not in the instructions, that check stops being possible, and the file quietly becomes a mixture of what the race says and what a lesson wanted.

A teaching waypoint is a property of the run, and the run file carries it as a `gate`. One course file, many runs, and the race still readable against its own authority.

AREAS 1

```
AREAS 1
source Invented for this document; a real file names its own source
layer coast-example-mhw
line
  50.60000 -1.95000
  50.60200 -1.94100
  …
end line
layer tss-example
area
  49.72000 -2.50000
  …
end area
end
```

Invented here too, and for the same reason as the course above. A real areas file's `source` line says which coastline at which datum, clipped and simplified how - which is what makes it checkable against the data it came from, and what an example in a document can never be.

Two shapes, because two things are being said

	what it means	the test
`line`	**must not be crossed**	does this step's segment cross it
`area`	**must not be entered**	does this step end inside it

The coastline stays as open lines, and is not closed into land polygons. practical-navigator asked. Closing a coastline means inventing which side is land — along every clipped edge and every seam between source tiles — and an invented boundary is one nobody can check against the source. The source data is lines; the file keeps them as lines; the test a router does against a coastline is a crossing test anyway.

A traffic separation scheme is an area because the rule is about being inside it, not about crossing a particular edge of it.

Layers, so a lesson can switch one off

Every shape belongs to a `layer`, named once and repeated for each shape under it. The run says which layers are live. That lets a lesson compare EMODnet against GSHHG by naming a different layer, with nothing else changing — which is the comparison practical-navigator asked for.

A layer name is free text with no spaces. The file does not say what a layer *means*; `source` does, in words, at the top.

Clearance is not here

practical-navigator asked whether each area should carry a safety margin. It should not, and the reason is that a margin is not a property of the water — it is a property of the boat and the day. A 2 m draught in a flat calm and the same boat in a four-metre sea want different distances off the same rock.

Clearance lives in the run file, per layer, so one set of areas serves every boat and every sea state. A legal boundary such as a TSS takes a clearance of zero, which is correct: you may touch its edge, you may not be inside it.

RUN 2

```
RUN 2
course  example_race_2025.txt
areas   areas_example_2025.txt
polar   polar_70ft_v1.txt
wind    wind_20250524_00z.grid
current current_20250524.grid
start   class Class One
step    time 600 heading 5
scale   wind 1.0 current 1.0
layers  coast-example-mhw tss-example boom-example
clear   coast-example-mhw 0.10  tss-example 0  boom-example 0
round   0.10
gate    Example Head 50.57868 -2.05644 within 2.0
avoid   Example Race 50.51000 -2.44000 off 3.0
end
```

RUN 1 is still read — `course.js` reads both, and a RUN 1 file keeps RUN 1's meaning exactly. What follows describes RUN 2; the two differences from RUN 1 are the grids and the gate, and each has its own section below.

Line	Form	Notes
`course`	a filename	The race.
`areas`	a filename	The water. May be repeated.
`polar`	a filename	**The boat is named here**, not in the course.
`wind`	a GRID filename	**Required in RUN 2.** The wind this run was sailed in.
`current`	a GRID filename	**Required in RUN 2.** The stream this run was sailed in.
`start`	`class <name>` or `at <time>`	By class from the course, or a time of your own.
`step`	`time <s> heading <deg>`	The isochrone step and the heading resolution.
`scale`	`wind <f> current <f>`	1.0 is as given. `current 0.5` is the OSCAR question.
`layers`	layer names	Which layers of the areas files are live.
`clear`	layer, distance in nm	Per layer. A layer not named takes 0.
`round`	distance in nm	How close counts as having rounded a mark.
`gate`	name lat lon `within` nm	Optional, repeatable. Pass **within** that distance of the point.
`avoid`	name lat lon `off` nm	Optional, repeatable. Stay **at least** that distance off it.
`end`	—	Last line.

The weather is part of the experiment, which is why RUN 2 exists

A run names its boat and its start, and until RUN 2 it did not name the two things that decide a route more than either: the wind and the stream. PILOT found it writing the 2025 run and kept them in a comment, which is the right answer to a format that cannot say something and the wrong place for them to stay.

Both are required. A route whose weather is not written down cannot be reproduced, and “which wind was that sailed in?” is the first question anybody asks of a surprising route.

Why a version and not another optional line. The rule below is that a new line is additive only if a reader that does not know it still reads a *correct* file. A reader that ignores `wind` does not: it routes on whatever grids it was handed and reports the result as this run — a convincing route in the wrong weather, which is the fault `GRID 1` exists to prevent. So it is RUN 2, and `course.js` keeps reading RUN 1.

The kind is checked against the grid, not against the filename. A filename is not evidence. `COURSE.checkGrids(run, grids)` compares each loaded grid's own `kind` header with the line that named it, so a current grid handed in as the wind is refused. It is a separate call because parsing text and loading files are different jobs, and the reader never opens a file.

A gate said two opposite things, so RUN 2 gives it two words

In RUN 1, `gate Example Head 50.57800 -2.05200 off 2.0` did not say which rule it meant, and the two readings are opposite instructions to a router:

	what it means
`gate <name> <lat> <lon> within <nm>`	the track must pass **within** that distance of the point — a teaching waypoint, which is how Larry's Level 1 route used it
`avoid <name> <lat> <lon> off <nm>`	the track must stay **at least** that distance off it — a keep-out circle

`within` and `off` are kept as different words on purpose, so a verb read too quickly cannot invert the rule. A RUN 1 gate keeps RUN 1's shape and is not given a meaning it never had.

On which side a headland is passed, the file says nothing, and does not need to. A keep-out circle plus the coastline in the areas file leaves only the seaward way round, because the landward side crosses the coast. Where a lesson wants a particular side anyway, that is what a `gate … within` is for: a point on the side you mean.

On order: the file order is the order, as it is for the course's marks, and a point passed twice — St Alban's Head going out and coming back — is written twice. A count or an index would be a second copy of what the file already says.

Why the boat and the start are both here. Larry asked for one course file that serves every boat and every start. That only works if neither is written into the course. A run is then exactly one experiment: this boat, this start, this strength of stream — and the lesson is the difference between two runs that share a course.

`scale` is what makes the lesson possible. `current 0.5` answers "what if the stream were half what the atlas says", which is the question a navigator actually has at a headland, and it costs one line rather than a second set of grids.

What the format cannot see, and why `source` is required

A position is checked for range: a latitude of 150 is impossible and is refused. **A pair that has been swapped into a position that is merely wrong is not detectable** — 4 degrees north and 50 degrees east are both perfectly legal numbers, and no format can tell that they were meant the other way round.

That is the honest limit, and it is the reason `source` is a required line. The defence against a wrong figure is not the reader; it is that the file looks like the sailing instructions and can be read against them. The reader catches what is impossible. A person catches what is merely untrue.

What all three refuse

The same discipline as GRID 1, for the same reason: a bad input to a router does not become an error, it becomes a *route*.

1. **An unknown keyword is refused, not ignored.** A line nothing reads is a line somebody believes.
2. **A missing `end` is refused.** That is the truncated-file check.
3. **A mark without a side, a line without a `cross`, a run naming a class the course does not list** — all refused, each with one plain sentence and the line number.
4. **A `clear` for a layer not in `layers`** is refused: it means somebody expected a margin on water the run is not looking at.

The reader never throws. A bad file is a result with a sentence and a line, in the same shape `GRID.parse` returns, because a reader that throws inside a router loses the route rather than reporting the file.

What COURSE 1 does not do

1. **No rhumb line or great circle choice.** Positions are positions; the router decides how it sails between them.
2. **No tidal gates, no time windows on marks.** When a race needs them it is COURSE 2, and course files written to this version still read.
3. **No scoring.** Whether a track sailed the course is the router's question; this file only says what the course is.

Changing them

A new optional line on an existing version is not a version change if a reader that does not know it still reads a correct course — `gate` in RUN 1 was of that kind. Anything that changes the meaning of a line that already exists, or that a reader must understand to be correct, is **COURSE 2**, **AREAS 2** or **RUN 2**, and `course.js` keeps reading version 1.

RUN 2 is the first use of that rule. The grids had to be understood to read the file correctly, and the gate's `off` changed meaning, so neither could be an addition to RUN 1. RUN 1 files are still read, and still mean what they meant.

Where this is likely to be wrong

bashnav owns the format and has written it from what an isochrone router needs. practical-navigator holds the actual 2025 data and the sailing instructions, and will find things this does not cover — a line stated some third way, a mark with a condition, an exclusion zone that is really a corridor. **Those are corrections to this document, not workarounds in the data.** Send them and the format changes.